Chương 7

Shell Programming

Nội dung

1. Shell Script
2. Các cấu trúc điều khiển
3. Phụ lục: Lập trình C trên Linux

1. Shell Script

- **Tạo 1 file script**: `vi <ten_file>`

  Ví dụ: Tạo file **bt01.sh** `vi bt01.sh`

- **Run file script**: *(Có nhiều cách - SV tự tìm hiểu thêm các cách khác)*.
  - **B1: Cấp quyền thực thi**.

    chmod +x ***tên_file***
  - **B2: Run file**.

    ./tên_file

1. Shell Script

- **Viết lệnh trong file script**:
  - Phân biệt ký tự **thường**, ký tự **HOA** (**ví dụ**: **no** khác **No**)
  - Dòng đầu tiên trong script file xác định loại **shell** đang sử dụng.

    Thử 1 trong 2 cách dùng bash shells:

    `#!/bin/bash`

    `#!/usr/bin/env bash`
  - Muốn dùng 2 hay nhiều lệnh trên 1 dòng

    → phân cách các lệnh bằng dấu '**;**'

  Ví dụ: **# ls; who; cal**

1. Shell Script

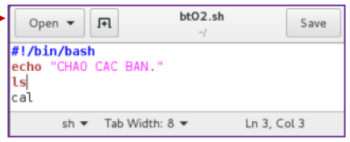
- **Một số lưu ý khi tạo shell script**:
  - Cú pháp lệnh
  - Ghi chú: dấu #
  - Khai báo biến
  - Các lệnh nhập, xuất
  - Phép gán
  - Biểu thức và Các phép toán số học
  - Các phép toán so sánh
  - Cấu trúc rẽ nhánh
  - Cấu trúc lặp
  - Các lệnh dừng/thoát

Ví dụ: tạo và thực thi 1 script file

- Tạo script file có tên **bt02.sh** bằng **gedit**

  # gedit bt02.sh
- Nhập các lệnh:

```
#!/bin/bash
# mot so lenh he thong
echo "Chao cac ban"
ls
cal
```



- **Save** → Đóng file.
- Cấp quyền thực thi: **# chmod +x bt02.sh**
- Thực thi file: **# ./bt02.sh**

1.1. Xuất thông báo ra màn hình

- Lệnh **echo**
  - **Xuất thông báo** rồi **xuống dòng**:

    echo *" Nội dung thông báo "*

  - **Xuất thông báo** nhưng **không xuống dòng**:

    echo -n *" Nội dung thông báo "*

1.2. Đọc thông tin từ bàn phím

- Lệnh **read** Dùng để nhập thông tin từ bàn phím.
- **Cú pháp**: **read** ***tên_biến***
- Việc nhập **kết thúc bằng phím Enter**.
- Nội dung nhập được chứa trong biến ***tên_biến***.
- **Ví dụ**:

  read N # dữ liệu chứa trong biến **N**

1.3. Sử dụng giá trị của biến

- Dùng dấu **$** trước tên biến khi muốn sử dụng giá trị của biến
- **Ví dụ**:

```
ten="MINH"
echo "Ban ten la $ten"
```

- Kết quả xuất ra màn hình nội dung:

  Ban ten la MINH

1.3. Sử dụng giá trị của biến

- **Ví dụ**: tạo file **bt0.sh** chứa các lệnh sau:

```
num1=12
num2=$(($num1 + 1))          # phải có 2 dấu ngoặc
num3=$[$num2 + $num1]
echo $num3
```

Run file → Kết quả xuất ra số: **25**

1.4. Một số biến môi trường

- HOSTNAME: Tên máy Linux
- HOME: Thư mục chủ của người dùng
- SHELL: Shell hiện hành
- PWD: Thư mục hiện hành
- PATH: Danh sách các thư mục để tìm kiếm các lệnh, phân cách bởi dấu hai chấm (**:**)
- USER: Tên người dùng hiện hành
- TERM: Kiểu của terminal hiện hành
- PS1: Dấu nhắc shell

1.5. Các phép toán số học

- **Các phép toán số học**
- **Các biểu thức số học**

1.5.1. Các phép toán số học

- Phép cộng: +
- Phép trừ: -
- Phép nhân: * **hoặc** *
- Phép chia nguyên: /
- Phép lấy dư: %

1.5.2. Các biểu thức số học

- Có nhiều cách tính biểu thức toán học:

Cách 1: let

Cách 2: $(( ))

Cách 3: $[ ]

Cách 3: expr ## ???

1.5.2. Các biểu thức số học

- **Ví dụ**:

```
let "sum = 4 + 3"              # cách 1
area=$(($length * $width))     # cách 2
percent=$[ $num / 100 ]        # cách 3
remain=$[ $n % $d ]            # cách 3
count=`expr $count + 1`        # cách 3 – test lai
```


1.6. Biểu thức điều kiện

- Là biểu thức kiểm tra logic, trả về kết quả là đúng (**1**) hay sai (**0**).
- Nó thường được dùng trong các cấu trúc điều khiển chương trình như: **if**, **while**,…
- Trong **shell script** sử dụng lệnh **[ ]** hoặc **test** để kiểm tra biểu thức logic.
- **Cú pháp**:

Cách 1: test *<biểu thức>*

Cách 2: [*<biểu thức>*]

1.6. Biểu thức điều kiện

- **Ví dụ**: Kiểm tra **a** lớn hơn **b**

```
if test $a –gt $b      # cách 1
then
<lệnh>
fi   # end if
```

Sử dụng **[ ]** thay cho **test**:

```
if [ $a –gt $b ]       # cách 2
then
<lệnh>
fi   # end if
```


1.6. Biểu thức điều kiện

- Các biểu thức điều kiện có thể là:
  - So sánh trên **chuỗi**
  - So sánh trên **số**
  - Kiểm tra trên **tập tin**
  - Kiểm tra trên **thư mục**

SỬ dụng tool: bash calculator

- **bash calculator** cho phép tính toán các phép toán số học (tương tự calculator)
- Lệnh sử dụng: # **bc**
- **Ví dụ:**

```
[hvn@hvnpc ~]$ bc
bc 1.06.95
Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006 Free Software Foundation,
nc.
This is free software with ABSOLUTELY NO WARRANTY.
For details type `warranty'.
5+1
6
15-5
10
12*3
36
20/4
5
quit
```


So sánh trên chuỗi

- **S1 = S2** Chuỗi S1 bằng chuỗi S2 không?
- **S1 != S2** Chuỗi S1 khác chuỗi S2 không?
- **S1 > S2** S1 đứng trước S2 theo thứ tự a, b không?
- **S1 < S2** S1 đứng sau S2 theo thứ tự a, b không?
- **-n S1** Chuỗi S1 có độ dài lớn hơn 0 không?
- **-z S1** Chuỗi S1 có độ dài bằng 0 không?

So sánh số học

- n1 **-eq** n2 n1 bằng n2 ?
- n1 **-ge** n2 n1 lớn hơn hoặc bằng n2 ?
- n1 **-gt** n2 n1 lớn hơn n2 ?
- n1 **-le** n2 n1 bé hơn hoặc bằng n2 ?
- n1 **-lt** n2 n1 bé hơn n2 ?
- n1 **-ne** n2 n1 khác n2 ?
- n! phủ định của biểu thức n (phép not)

Kiểm tra trên tập tin, thư mục

- **-f file** **file** có phải là tập tin không.
- **-d dir** **dir** có phải là thư mục không.
- **-c name** **name** có phải là tập tin ký tự không.
- **-s name** **name** có kích thước lớn hơn 0 hay không

2. Các cấu trúc điều khiển

1. Cấu trúc rẽ nhánh **if**
2. Cấu trúc lựa chọn **case**
3. Cấu trúc lặp **for**
4. Cấu trúc lặp **while**
5. Cấu trúc lặp **until**

2.1. Các phép toán logic dùng trong điều kiện

- Phép **AND**: **&&**
- **Ví dụ:**

```
if [ $base -eq 1 ] && [ $dm -eq 1 ]
then
        echo "Wow !"
fi
```

- Phép **OR**: **||**
- **Ví dụ:**

```
if [ $base -eq 1 ] || [ $dm -eq 1 ]
then
        echo "Ohhh !"
fi
```

Xem lại các phép so sánh chuỗi và số

String	Numeric	True if
x = y	x -eq y	x is equal to y
x != y	x -ne y	x is not equal to y
x < y	x -lt y	x is less than y
x <= y	x -le y	x is less than or equal to y
x > y	x -gt y	x is greater than y
x >= y	x -ge y	x is greater than or equal to y
-n x	–	x is not null
-z x	–	x is null

2.2. Cấu trúc rẽ nhánh if

Cú pháp:

```
if [ điều kiện ]
then
   <nhóm lệnh>
[elif [ điều kiện ]
then
   <nhóm lệnh> ]
...
[else
   <nhóm lệnh> ]
fi   # end if
```

Ví dụ:

```
echo "Enter value:"
read num
if [ $num –lt 9 ]
then
   echo "$num nho hon 9"
else
   echo "$num lon hon 9"
fi  # end if
```


2.2. Cấu trúc rẽ nhánh if

❖ Ví dụ:

```
#!/bin/bash
echo –n "Nhap so a: "
read a
echo –n "Nhap so b: "
read b
if [ $a -lt $b ]
then
   echo "So $a nho hon so $b."
elif [ $a -eq $b ]
then
   echo "So $a bang so $b."
else
   echo "so $a lon hon so $b."
fi   # end if
```


Bài tập:

Bài 1: Viết shell script thay đổi giá trị của 2 số: a, b

Bài 2: Viết shell script tìm giá trị lớn nhất trong ba số: a, b, c.

Bài 3: Viết shell script giải phương trình bậc nhất: ax + b = 0.

2.3. Cấu trúc lựa chọn case

❖ Cú pháp:

```
case $<biểu thức> in
  <giatri1>)
     <nhóm lệnh1>;;
  < giatri2>)
     <nhóm lệnh2>;;
  *)
     <nhóm lệnh#>;;
esac   # end case
```

* Ví dụ:

```
echo "Nhap 1 so:"
read number
case $number in
  1)
     echo "One";;
  2)
     echo "Two";;
  *)
     echo "Unknown";;
esac   # end case
```


2.3. Cấu trúc lựa chọn case

Ví dụ:

```
#!/bin/bash
echo "Is it morning? Please answer yes or no:"
read answer
case "$answer" in
  "yes")
      echo "Good morning";;
  "no")
      echo "Good afternoon";;
  *)
      echo "Sorry, answer not recognized.";;
esac   # end case
```

Bài tập

Bài 1: Viết shell script thực nhập vào hai số và một toán tử sau đó hiển thị kết quả.

Bài 2: Viết shell script tính điểm trung bình và xếp loại của một thí sinh thi tuyển sinh (có ba cột điểm)

Bài 3: Viết shell script thực hiện nhập vào môt giá trị là thứ trong tuần. Hiển thị chữ tiếng anh của thứ đó.

2.4. Cấu trúc while

❖ Cú pháp:

```
while <điều_kiện>
do
    <các lệnh>
done
```

❖ Ví dụ:

```
#!/bin/bash
INT=1
while [ $INT  -lt  5 ]
do
   echo $INT
   INT=$(($INT+1))
done   # end while
```


2.4. Cấu trúc while

Ví dụ:

```
echo  "Chuong trinh tinh tong tu 1 den n"
echo  -n  "Nhap n =  "
read  n
index=0
tong=0
while [ $index -lt  $n ]
do
        index=$(($index + 1))
        tong=$(($tong + $index))
done   # end while
echo  "Tong tu 1 den n = $tong"
exit 0
```

Chạy chương trình :

```
#chmod a+x tong.sh
#./tong.sh
```


2.4. Cấu trúc while

Ví dụ:

```
#!/bin/bash
echo "Chuong trinh dem so dong cua tap tin $1"
{
n=0
while read line
        do
                n=$(($n + 1))
        done    # end while
echo "So dong cua tap tin la: $n"
}
exit 0
```

Chạy chương trình :

```
chmod a+x demdong.sh
./demdong  bai1.txt
```


2.5. Cấu trúc until

❖ Cú pháp:

```
until <điều_kiện>
do
    <các lệnh>
done
```

❖ Ví dụ:

```
#!/bin/bash
INT=1
until [ $INT -gt  4 ]
do
   echo $INT
   INT=$(($INT+1))
done   # end until
```


2.6. Cấu trúc for

❖ Cú pháp:

```
for  X  in  <list> ;
do
   <Các lệnh>
done
```

❖ Ví dụ:

```
#!/bin/bash
for  X  in  dog  cat  mouse ;
do
   echo "$X"
done   # end for
```

2.6. Cấu trúc for

❖ **Ví dụ:**

```
for gt in apple banana 34;
do
    echo $gt
done    # end for
```

```
Output:    apple
           banana
           34
```

TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh

Bài tập

1. Viết chương trình nhập vào số nguyên dương N.
   + Tính và xuất ra tổng các số dương lẻ
   + Tính và xuất ra tổng các số dương chẵn
2. Viết chương trình nhập vào số nguyên dương N.
   Xác định N có là số nguyên tố hay không?
3. Viết chương trình nhập vào số nguyên dương N
   Tìm và hiển thị tất cả các số nguyên tố từ 1 đến N
4. Viết chương trình nhập vào số nguyên dương N.
   Tính và xuất ra màn hình tổng của các số nguyên tố từ 1 đến N.
5. Viết chương trình nhập vào số nguyên dương N. Tính tổng các ước số của N

TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh

Phụ lục: Lập trình C trong linux

1. Soạn thảo chương trình. Lưu tập tin với tên và phần mở rộng thích hợp.
   # vi example.c
2. Từ dấu nhắc lệnh, gõ lệnh biên dịch :
   Cú pháp : **# gcc -o filedestination filesourcecode**
   Ví dụ: **# gcc -o example example.c**
3. Nếu có lỗi, trình biên dịch sẽ thông báo số thứ tự dòng lệnh lỗi.
   Nếu biên dịch thành công → chạy chương trình:
   Cú pháp chạy: **#./filedestination**
   Ví dụ: **#./example**

TRƯỜNG ĐẠI HỌC VĂN HIẾN
Thành nhân trước thành danh

Phụ lục: Lập trình C trong linux

1. Chương trình **hello.c**

```
#include <stdio.h>
main() {
    int i;
    for(i=1; i<11; i++) {
        printf("%2d\n", i);
    }
}
```